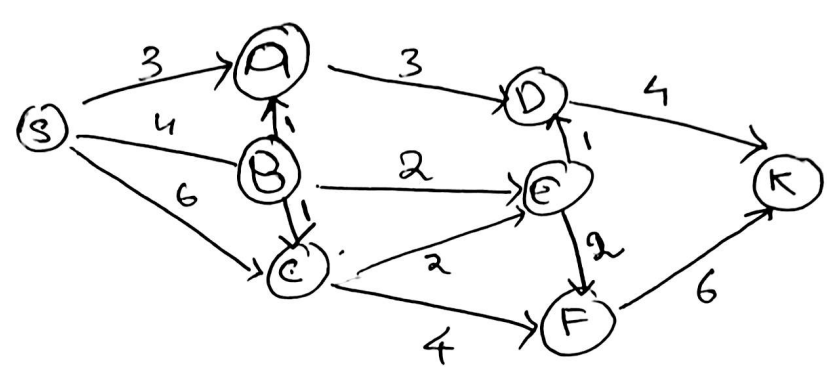


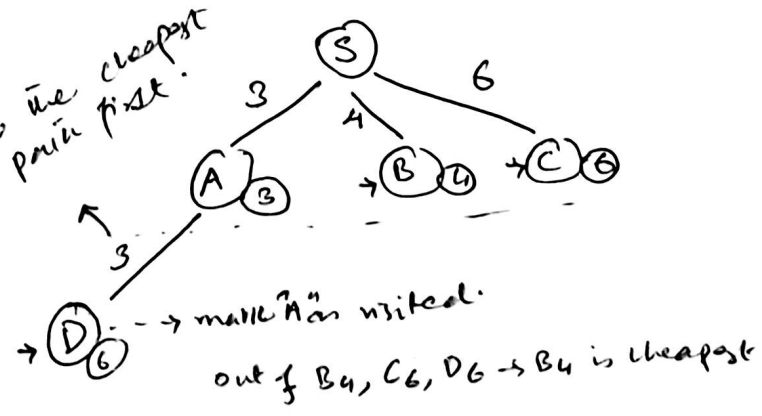
Uniform cost search/cheapest first search.

- uninformed search
- based on counting a visited
- Variant of Dijkstra SP



goal is $S \rightarrow K$.

expand the cheapest pair first.

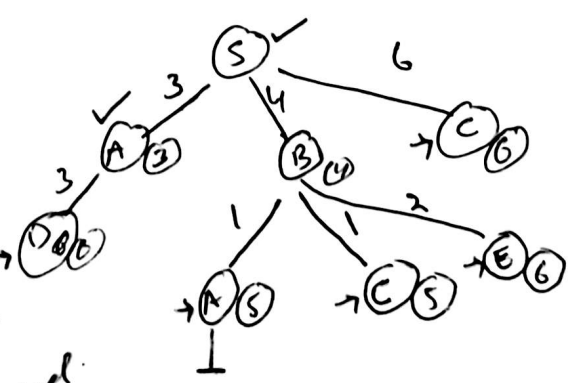


visited:
S

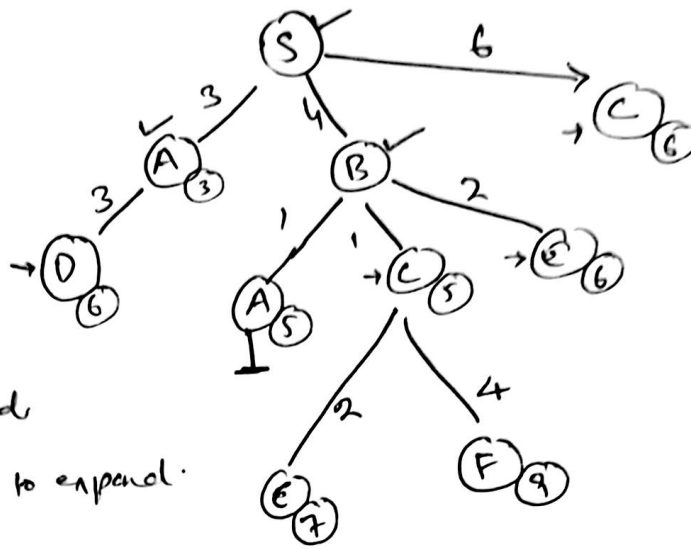
visited
S, A

Active

D_6, A_5, C_6, E_6
As no expansion as already visited - we make C5 & make B as visited.



visited
S, A, B

visited

S, A, B, C

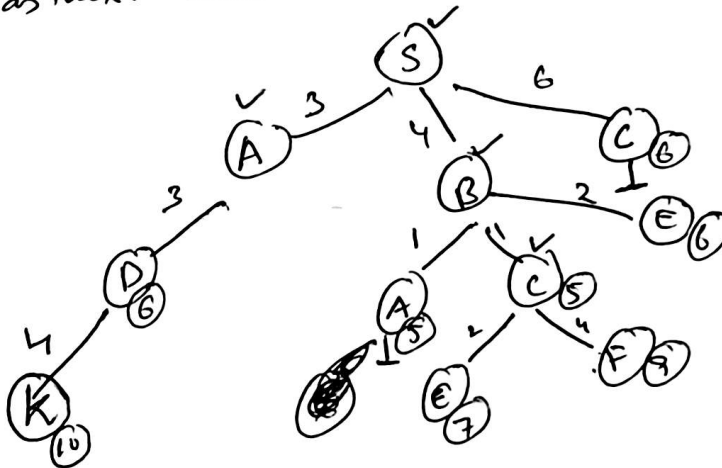
- Mark "C" as visited
- available nodes to expand:

D₆, E₇, F₉, E₆, C₆

Since "C" already visited with lower cost ($5 < 6$), mark as no further expansion.

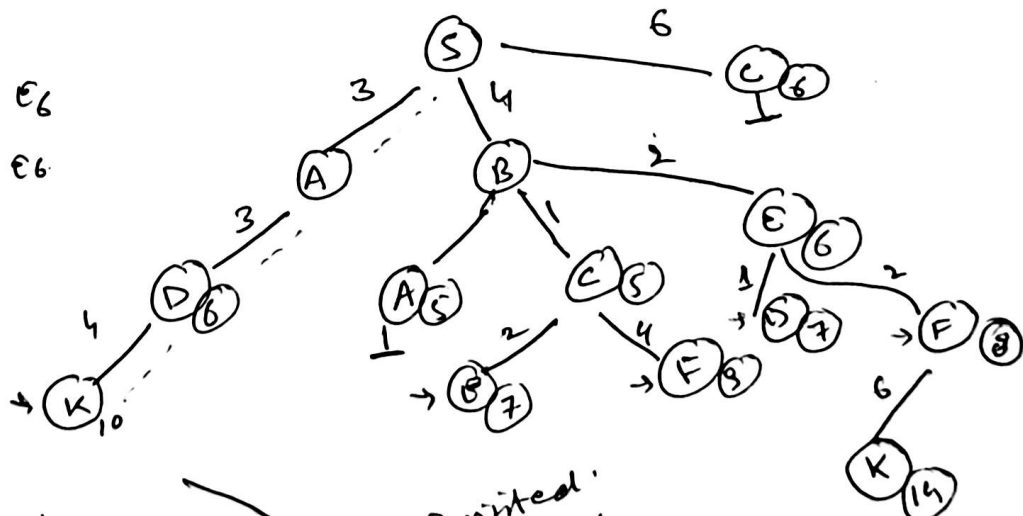
take D₆ as next node.

→ mark "D" as visited.

visited

S, A, B, C, D.

- next nodes to expand:

K₁₀, E₇, F₉, E₆take E₆visited

S, A, B, C, D, E

- E visited
- nodes to expand:

K₁₀, E₇, F₉, D₇, F₈
 E₉ D already visited.
 K₁₀, F₉, F₈

mark F visited.
 nodes to expand:
 K₁₀, K₁₄, F₉
 → F already visited

take K₁₀ ∴ K₁₀ is goal node
 the path is S → A → D → K

visited
 S, A, B, C, D, E, F

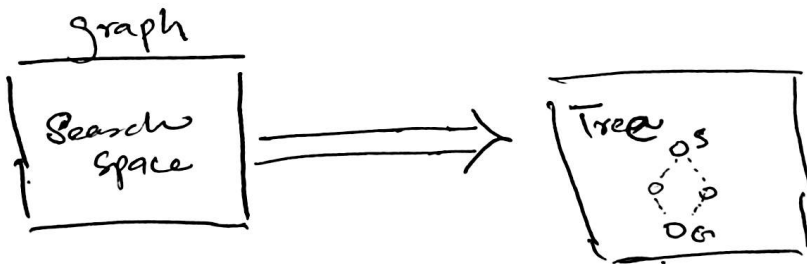
Breadth First Search.

(1)

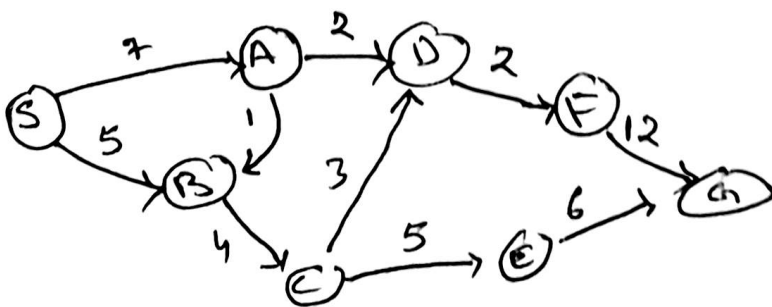
It is an

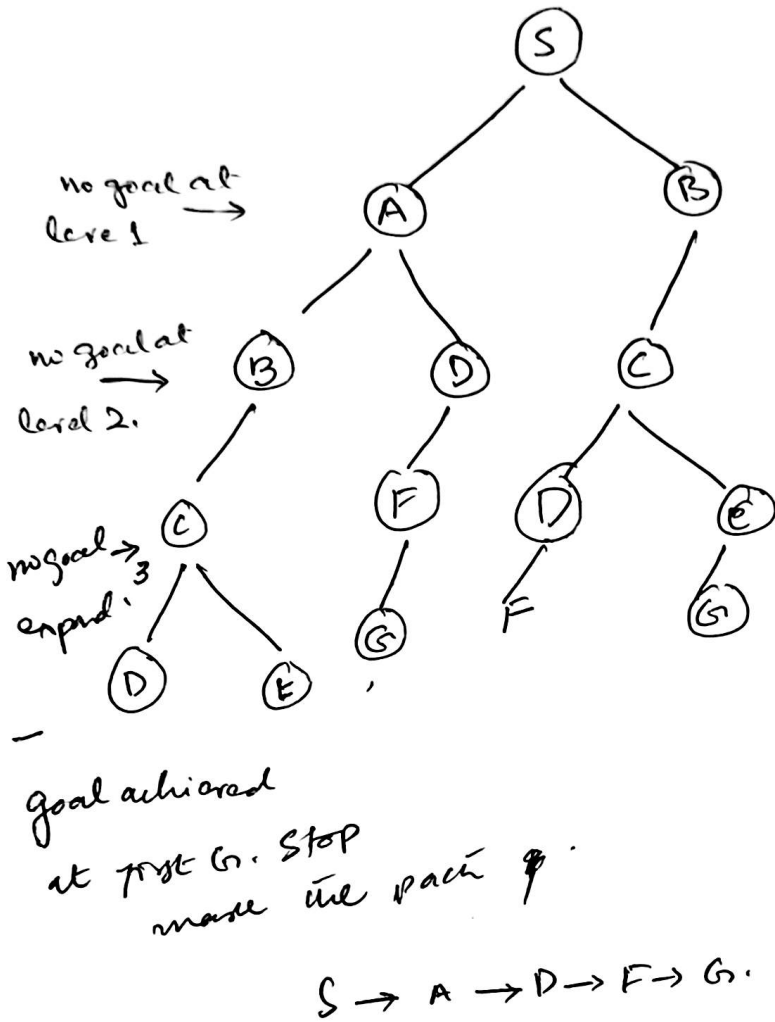
Uninformed Search.

↳ Bfs First Search finds the shortest path.
the path having less possible arcs/links/edges.



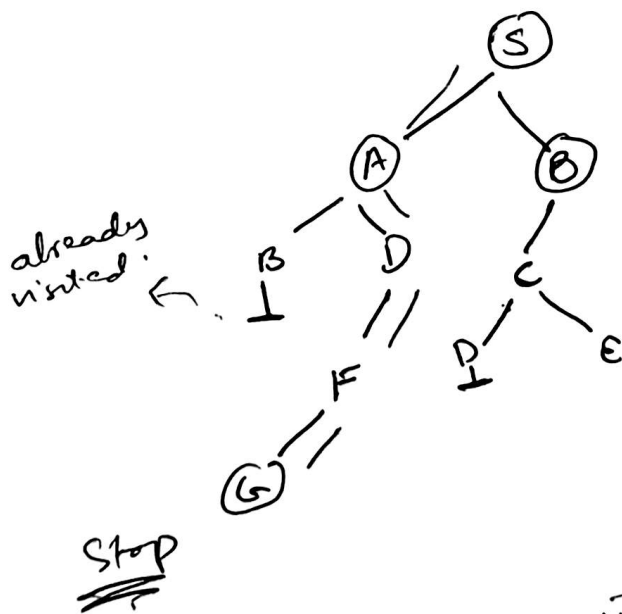
- The idea is that we start at "S" or initial state.
- Visit all directly connected, just in any order.
- look if goal achieved at level 1. stop if Yes
- No, continue expanding the nodes in level 1 (take left → right) for simplicity
- expand all nodes in level 1 from L-R & you end up in level 2.
- look if goal achieved at level 2, stop if Yes
- No, continue expanding nodes in level 2, from L-R.
- & you repeat.
- exhaustive, does not look at edge costs.





2nd Variant of Bd First Search.

Add memory to agent as states visited



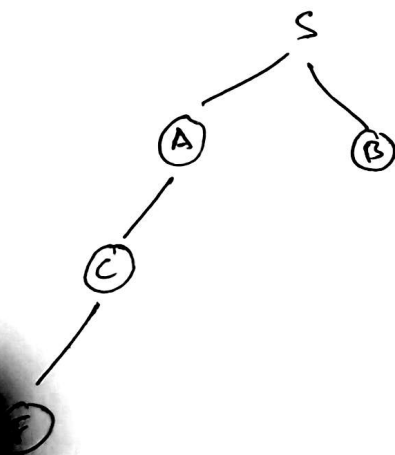
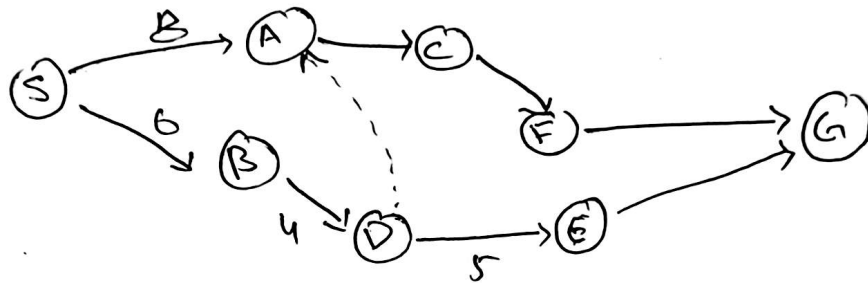
visited
S.

→ make A level 1
S, A
make B as visited level 1
S, A, B.
make D visited level 2
S, A, B, D.
make C visited level 2
S, A, B, D, C.
make the path /
 $S \rightarrow A \rightarrow D \rightarrow F \rightarrow G$

Depth First Search.

(1)

Uniformed Search.



put on stack.

visited.

$\begin{bmatrix} A \\ B \end{bmatrix}$

S

$\begin{bmatrix} C \\ B \end{bmatrix}$

S, A.

$\begin{bmatrix} F \\ B \end{bmatrix}$

S, A, C

$\begin{bmatrix} G \\ B \end{bmatrix}$

S, A, F

S, A, F, G.

The visited list is maintained to avoid loop.

If you search/find a node that is already in the visited list, you don't expand it to avoid loop.

Not the best algo, but has lower memory than Breadth First Search.

Searching in AI Paradigm.

1. is informed search.
2. $\hookrightarrow$ Uninformed search.

(1) Informed Search.

$\hookrightarrow$ A* Search algo

A* $\rightarrow$ based on heuristic, ^{prior} info about reaching to goal.
heuristic ~~is based on~~ are ^{are} ~~feel~~ ^{feel} ~~ments~~ ^{ments} a function "f"

$$f(n) = g(n) + h(n)$$

$\hookrightarrow$ any node in the graph (let assume not the start or goal node).



Actual cost of the total path from start to this node.

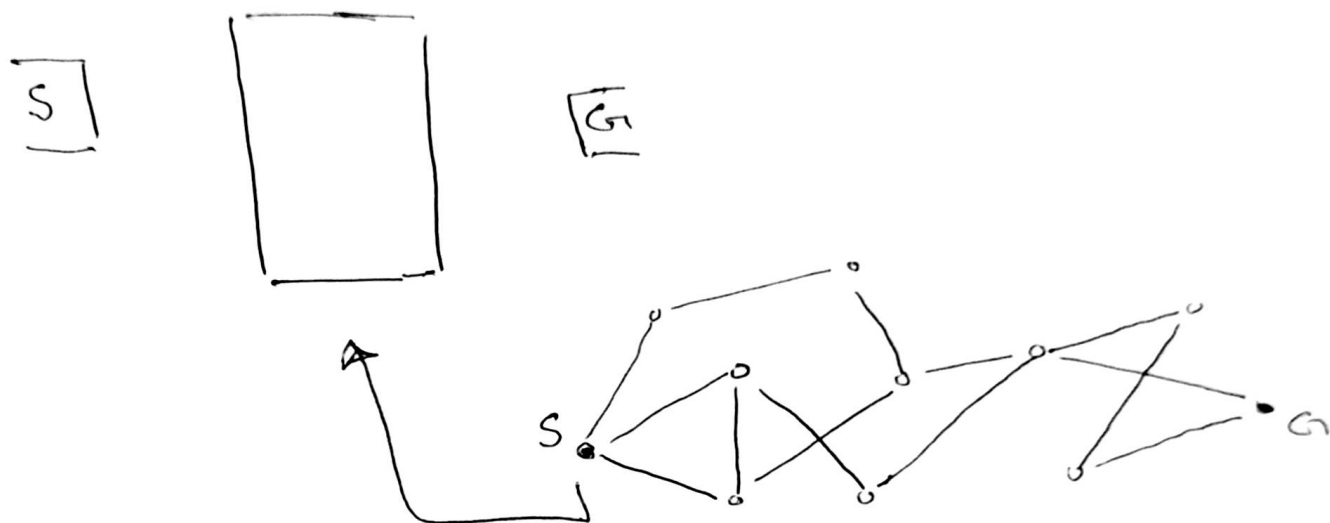
S $\rightarrow$ start node
n $\rightarrow$ intermediate node
G $\rightarrow$ goal, final destination

$\hookrightarrow$ cost of node for the whole path originating from itself to goal state

A $\rightarrow$ called as $g(n)$

B $\rightarrow$ called as $h(n)$

f $\rightarrow$ total cost from start node to goal node via "n"



Here we roughly divide our graph into 3 different layers.

$\begin{array}{c} | \quad | \quad | \quad | \quad | \\ S \quad \boxed{\text{Intermediate}} \quad G \end{array}$

$S \rightarrow$ start
 Intermediate node
 $G \rightarrow$ goal.

Result will be a large matrix of paths.

Path is set of nodes originating from "S" ending at G .

Sequence of intermediate node connecting to "G" eventually. might have loops as well

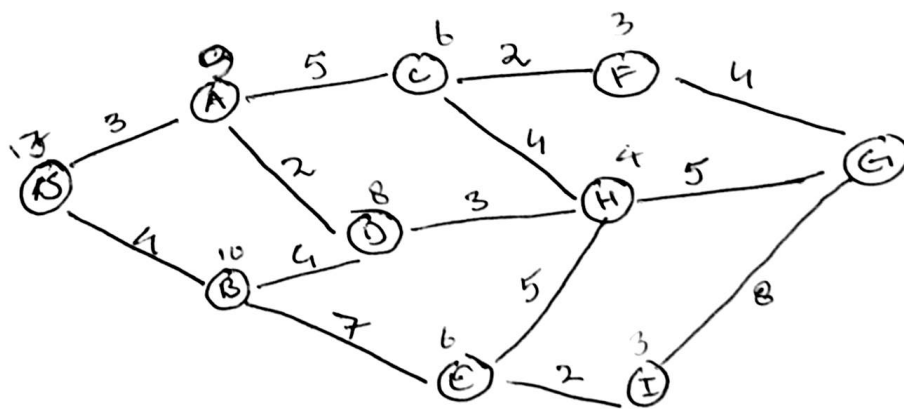
S						G
S						G
S						G
S						G

Our final goal is select optimum one, the one with min-cost $\rightarrow$ objective function

lets discuss about heuristic cost.

previously we used notation $\text{cost}(s, a, s')$

Shall means cost of the agent when action "a" is performed by changing the current state "s" to state "s'" - lets call it here (only here) the heuristic function applied on every node reaching to goal.



Uniform
cost Search
?

→ two cost functions.

- written on link
- (1) The actual cost of connected link $g(n)$
 - (2) heuristic cost of the any node reaching to goal $h(n)$

heuristic should never over-estimate i.e. should be less than actual cost.

$$h(S, G) = 17$$

$$h(A, G) = 12$$

$$h(B, G) = 10$$

$$\vdots$$

$$\vdots$$

$$h(G, G) = 0$$

written on top of every node.

lets perform A* on graph.

(4)

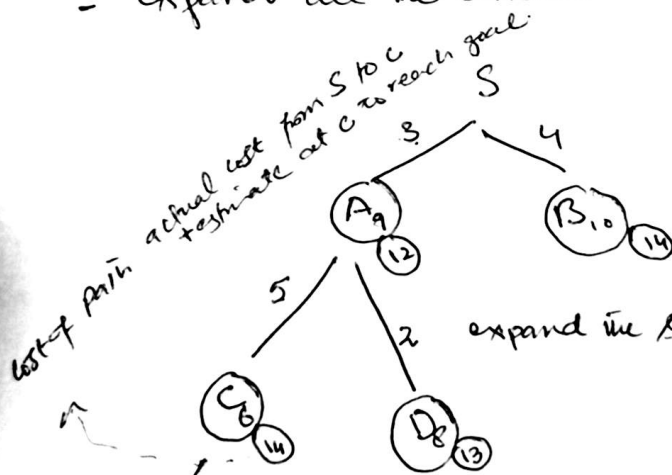
You maintain a list of visited states. call it "Visited"

- start with initial state.
"S"

Visited

S13

- expand all the children



expand the smaller one.

→ add A12 to visited

S13, A12

Here we have 3 diff scores (14) (13) & (14) take any (14)

(2 shortest)

(C8)

now C is visited add to the list

C8 is visited with A score of 4

S13, A12, C14

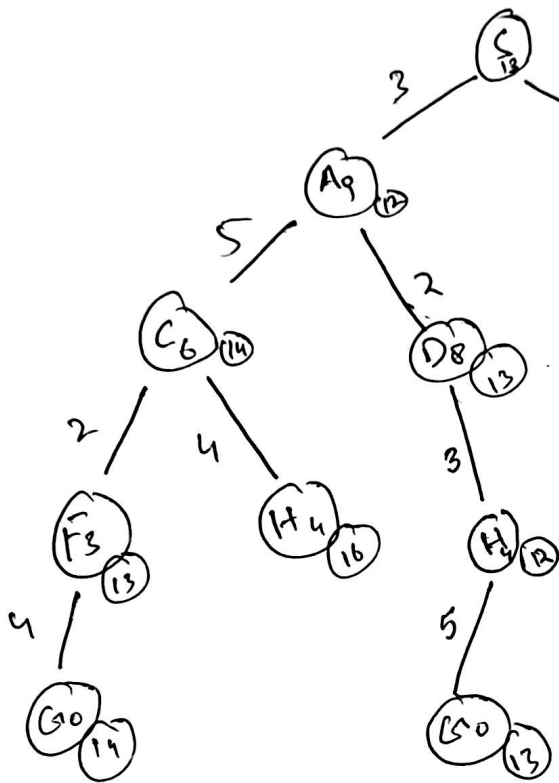
now got (13) (16) (13) (14) take smallest (13)

& expand

add F13 to visited

S13, A12, C14, F13

what are left: H16, D13, B14 expand D13



add D13 to list.

S13, A12, C14, D13

left nodes are H4, H12 & B10
expand the lowest H4

Add H4 to list

wroteed

S13, A12, C14, D13, H12

left nodes:

B14 & H16.

expand B14

You would end up with multiple goals from

S. take the one with lowest A* cost &

mark the path as optimum.